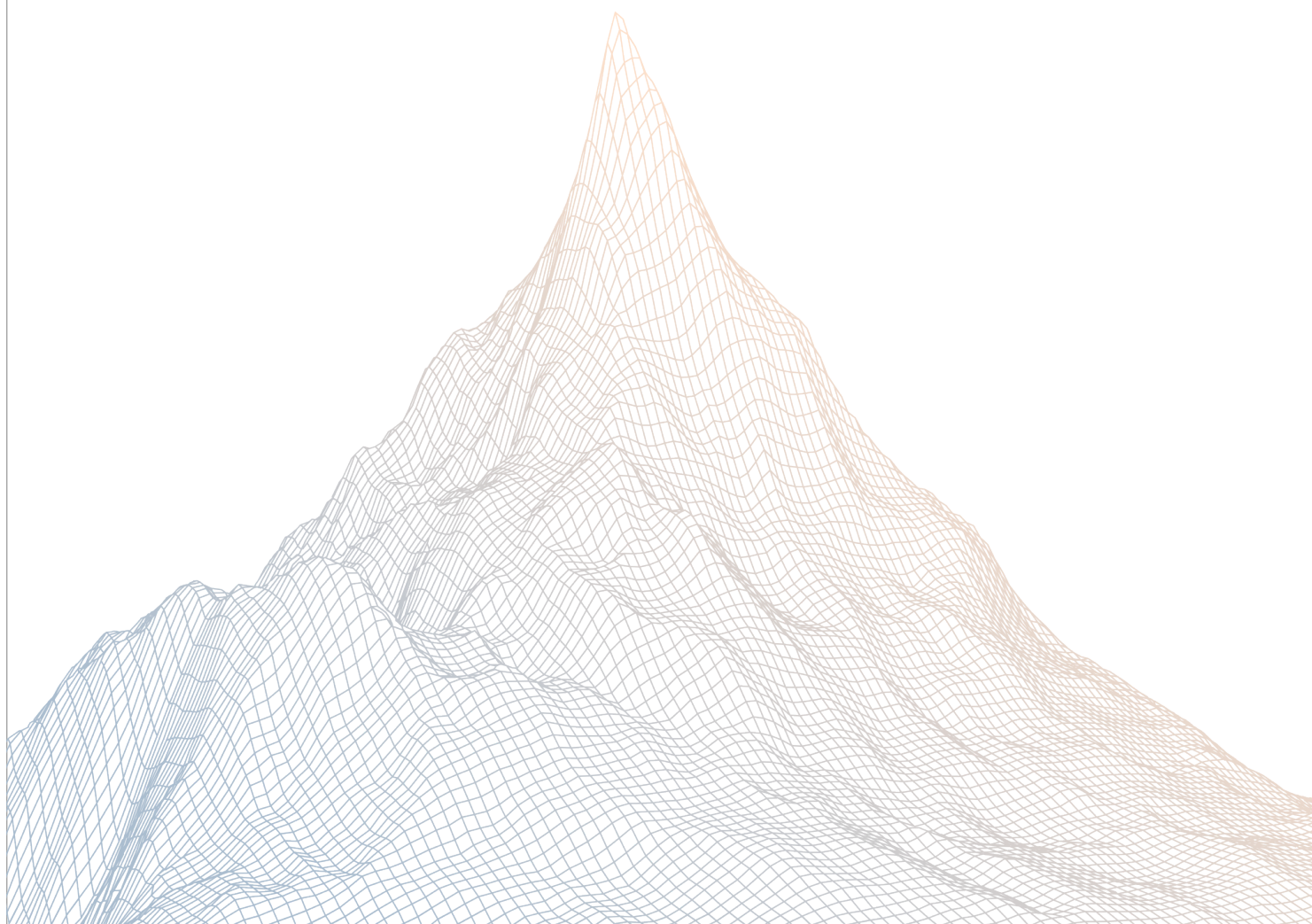


Meteora

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES:

November 7th to November 12th, 2025

AUDITED BY:

peakbolt

Contents

1	Introduction	2
1.1	About Zenith	3
1.2	Disclaimer	3
1.3	Risk Classification	3
2	Executive Summary	3
2.1	About Meteora	4
2.2	Scope	4
2.3	Audit Timeline	5
2.4	Issues Found	5
3	Findings Summary	5
4	Findings	6
4.1	Low Risk	7
4.2	Informational	9

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Meteora

Our mission is to build the most secure, sustainable and composable liquidity layer for all of Solana and DeFi.

By using Meteora’s DLMM and Dynamic AMM Pools, liquidity providers can earn the best fees and yield on their capital.

This would help transform Solana into the ultimate trading hub for mainstream users in crypto by driving sustainable, long-term liquidity to the platform. Join us at Meteora to shape Solana's future as the go-to destination for all crypto participants.

2.2 Scope

The engagement involved a review of the following targets:

Target	damm-v2
Repository	https://github.com/MeteoraAg/damm-v2
Commit Hash	122677027a44afbb0ffb4faf9e36cdb44a71a542
Files	Diff from 2d6c65d56e122416d810b79114348c8cdfaa0985

2.3 Audit Timeline

November 7, 2025	Audit start
November 12, 2025	Audit end
November 12, 2025	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	1
Informational	1
Total Issues	2

3

Findings Summary

ID	Description	Status
L-1	Pool::get_max_amount_in() incorrectly uses get_delta_amount_a_unsigned_unchecked() for TradeDirection::BtoA	Acknowledged
I-1	Use unsigned_abs() instead of abs() as u32	Acknowledged

4

Findings

4.1 Low Risk

A total of 1 low risk findings were identified.

[L-1] `Pool::get_max_amount_in()` incorrectly uses `get_delta_amount_a_unsigned_unchecked()` for `TradeDirection::BtoA`

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [pool.rs#L1125-L1145](#)

Description:

In `Pool::get_max_amount_in()`, it incorrectly calls `get_delta_amount_a_unsigned_unchecked()` instead of `get_delta_amount_b_unsigned_unchecked()` for the direction `TradeDirection::BtoA`.

This will cause it to return an incorrect max amount in for the direction B to A.

Recommendations:

This can be resolved as follows:

```
pub fn get_max_amount_in(&self, trade_direction: TradeDirection) ->
    Result<u64> {
    let amount = match trade_direction {
        TradeDirection::AtoB => get_delta_amount_a_unsigned_unchecked(
            self.sqrt_min_price,
            self.sqrt_price,
            self.liquidity,
            Rounding::Down,
        )?,
        TradeDirection::BtoA => get_delta_amount_a_unsigned_unchecked(
        TradeDirection::BtoA => get_delta_amount_b_unsigned_unchecked(
            self.sqrt_price,
```

```
        self.sqrt_max_price,  
        self.liquidity,  
        Rounding::Down,  
    )?,  
};  
if amount > U256::from(u64::MAX) {  
    Ok(u64::MAX)  
} else {  
    Ok(amount.try_into().unwrap())  
}  
}
```

Meteora: Acknowledged. The function `get_max_amount` is only called in test files, so we'll leave as is in the current version.

4.2 Informational

A total of 1 informational findings were identified.

[I-1] Use `unsigned_abs()` instead of `abs()` as `u32`

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [fee_math.rs#L46](#)

Description:

The `pow()` will perform a `exp.abs()` as `u32` when `exp` is negative.

However, it could result in an incorrect result when `exp = i32::MIN`, as the absolute value of `i32::MIN` cannot be represented as an `i32` and it could either trigger a panic or return `i32::MIN` as described [here](#).

Though this issue has no impact now as there are no instances where `exp` is negative when calling `pow()`.

Recommendations:

This can be resolved by using `unsigned_abs()` instead as follows:

```
pub fn pow(base: u128, exp: i32) -> Option<u128> {  
    ..  
  
    // Make the exponential positive. Which will compute the result later by  
    1 / base^exp  
    let exp: u32 = if invert { exp.abs() as u32 } else { exp as u32 };  
    let exp: u32 = if invert { exp.unsigned_abs() } else { exp as u32 };
```

Meteora: Acknowledged. We'll leave as is for now and update in a next release version.